Teams Chat & Channel DLP — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Teams Chat & Channel DLP in Microsoft Purview 4
- The problem . . . 4
- The solution . . . 4
- How it works — the model . . . 4
- What you get . . . 4
- Scope and boundaries . . . 5
- The bottom line . . . 5

Solution Overview — Teams Chat & Channel DLP in Microsoft Purview 5
- What this solution delivers . . . 5
- The two coverage areas (how Teams DLP actually maps) . . . 6
- What Teams message DLP is (and is not) . . . 6
- Simulate first — these are high-impact controls . . . 6
- The standard deployment model . . . 7
- What's in the box . . . 7

Requirements — Teams Chat & Channel DLP in Microsoft Purview 8
- R1. Licensing that includes Teams message DLP, assigned to in-scope users . . . 8
- R2. Least-privilege roles for DLP authoring . . . 8
- R3. Teams deployed with the in-scope chat/channel surfaces identified . . . 9
- R4. In-scope sensitive content to key the rules on (labels and/or SITs) . . . 9
- R5. All three DLP locations understood: Teams messages + SharePoint + OneDrive . . . 9
- R6. Message action + internal-vs-external decision — agreed with stakeholders . . . 9
- R7. A named pilot user group and change control (simulation/test first) . . . 10
- R8. Policy-tip / user-notification content prepared (where used) . . . 10

Global Requirements — Microsoft Purview Solutions 10
- 1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11
- 2. Purview roles (least-privilege) and portal access . . . 11
- 3. Security & Compliance PowerShell + Graph toolchain . . . 11
- 4. Data sources & scope in scope . . . 11
- 5. Compliance, records, privacy & legal stakeholders . . . 12
- 6. A pilot scope and change control . . . 12

Implementation Guide — Teams Chat & Channel DLP in Microsoft Purview 12
- Phase 0 — Assess (confirm you are ready) . . . 13
- Phase 1 — Design the policy (locations, rules, actions) . . . 13
- Phase 2 — Author the policy + rules in SIMULATION . . . 14
- Phase 3 — Pilot & validate in simulation . . . 15

Phase 4 — Enforce, then expand . . . 15
Rollback summary (fast reference) . . . 15
Testing & Validation — Teams Chat & Channel DLP 16
A. Configuration gates (automated — `DLPTEAMS-Validate.ps1`, read-only) . . . 16
B. Behavioural tests — messages and files (manual, as a pilot user) . . . 17
C. Negative / false-positive tests (both passes) . . . 18
D. Evidence to capture (for the defensibility pack) . . . 18
Troubleshooting — Teams Chat & Channel DLP 19
1. Messages are covered but shared FILES are not (the classic gap) . . . 19
2. The Teams-messages location isn't available, or message DLP never fires . . . 19
3. The rule fires on the wrong things (false positives) . . . 20
4. The rule doesn't fire on content it should . . . 20
5. The internal-vs-external logic behaves unexpectedly . . . 20
6. Simulation looks fine but enforcement doesn't block . . . 21
7. `New-DlpCompliancePolicy` / location parameter errors . . . 21
8. A default Teams DLP policy is interfering . . . 21
9. Activity report is empty or export fails . . . 22
10. "It blocked a legitimate conversation / share" — emergency rollback . . . 22
When to escalate to the determination owners (not a technical fix) . . . 22
Best Practices — Teams Chat & Channel DLP 22
Cover messages AND files — the single most common gap . . . 23
Deploy the way the house model says — simulate, then enforce . . . 23
Get the internal-vs-external logic right . . . 23
Key the rules on *sensitive* content, not everything . . . 23
Write a clear policy tip . . . 24
Operate it, don't set-and-forget . . . 24
Keep the roles and determinations where they belong . . . 24
Security & handling of the tooling . . . 24
Deliverables Summary — Teams Chat & Channel DLP in Microsoft Purview 25
Documents . . . 25
PowerShell tools (5) . . . 26
How it maps to the deployment model . . . 26
Definition of done . . . 27
Appendix A — PowerShell Scripts 28

Executive Summary — Teams Chat & Channel DLP in Microsoft Purview

The problem

Microsoft Teams is where a large share of real-time work — and sensitive data — now moves. A payment card number pasted into a **chat**, a client's health record dropped into a **channel post**, a confidential contract **shared as a file** in a team: these are everyday Teams actions, and without a control they flow to whoever is in the conversation, **including external guests and federated users**. Left unaddressed, that is both a **compliance gap** and a direct **data-risk exposure**.

The solution

This solution delivers **Microsoft Purview Data Loss Prevention for Teams** end to end, covering the two ways sensitive data moves through Teams:

- **Teams messages (chat + channel):** a DLP rule detects sensitive content in 1:1/group chats and channel posts, blocks the message with a **policy tip**, and applies differently to **internal** versus **external** recipients.
- **Files shared in Teams:** files shared in a **chat** live in the sender's **OneDrive** and files shared in a **channel** live in the team's **SharePoint** site — so the policy also spans **SharePoint** and **OneDrive** to govern those files. Enabling only the message location would leave shared files uncovered; this solution closes that common gap.

Crucially, every rule is **piloted in simulation/test mode first** — it records what it *would* do without blocking anyone — validated on a small user ring, and only then moved to **enforce**.

How it works — the model

The delivery follows a disciplined, repeatable model with an explicit pass/fail gate at each step:

Assess → **Design** → **Configure** → **Pilot (simulation)** → **Validate** → **Expand (enforce).**

Because a block on a Teams message or shared file interrupts real collaboration, nothing is enforced until it has been proven, in simulation, to see the right content and act only where intended — with a one-command rollback available at all times.

What you get

- A complete document set: Overview, Requirements, an ordered Implementation guide, Testing (pass/fail gates), Troubleshooting, Best Practices, and this summary.
- **Five PowerShell tools** (Security & Compliance PowerShell / Microsoft Graph) — discovery, configuration (simulation, `-WhatIf` first), validation, reporting, and scope/health — all **read-only or preview by default, with no credentials in code.**
- A **defensibility evidence pack**: proof of what was configured across messages **and** files, that

each path was validated in simulation before enforcement, and that enforcement (including internal-vs-external) behaves as designed.

Scope and boundaries

- Teams **message-level** DLP is **licensing-dependent** (a premium capability — verified against the

Microsoft Purview service description for your tenant) and the Purview portal and Teams DLP scope (e.g. private/shared channels) **evolve** — capability availability and exact paths are **verified against your licensing/tenant**, never assumed.

- **Regulatory, records, and legal determinations** — which data is sensitive, which paths must be

blocked versus audited, and whether a message or share is a violation — remain with your **compliance, security, records, privacy, and legal** functions. This solution provides the **technical method and the evidence**, not the determination.

The bottom line

Sensitive data moving through **Teams chats, channels, and shared files** becomes a **monitored, then enforced** control in Microsoft Purview — messages and files, internal and external — **configured, piloted in simulation, validated, and defensible.**

Solution Overview — Teams Chat & Channel DLP in Microsoft Purview

What this solution delivers

Microsoft Teams is where a great deal of an organization's real-time work now happens — and a great deal of its sensitive data. A credit-card number pasted into a **1:1 chat**, a client's health record dropped into a **channel post**, a confidential contract **shared as a file** in a team: these are everyday Teams actions, and without a control they flow to whoever is in the conversation — including **external guests and federated users**. Left unaddressed, that is both a **compliance gap** and a direct **data-risk exposure**.

This solution delivers **Teams chat & channel Data Loss Prevention** end to end in Microsoft Purview, covering the two ways sensitive data moves through Teams:

1. **Teams messages (chat + channel).** A DLP policy on the **Teams chat and channel messages**

location can **detect and act on** sensitive content in **1:1 and group chats**, **standard and private channel messages** — blocking the message, showing a **policy tip**, and (critically) applying differently to **internal** versus **external / everyone** recipients.

1. **Files shared in Teams.** Files shared in a Teams **chat** live in the sender's **OneDrive**, and

files shared in a **channel** live in the team's **SharePoint** site — so **SharePoint Online** and **OneDrive** DLP is what governs Teams-shared files. This solution configures those locations in the same policy so a "Teams DLP" control actually covers the files too, not just the message text.

The result is *stop sensitive data in Teams messages and files* — configured, **piloted in simulation/test** first, validated, and defensible.

This solution walks a competent-but-novice administrator through the complete, repeatable deployment:

1. **Confirm licensing, roles, and in-scope sensitive content** — Teams message DLP is licensing-dependent; the content to protect (labels / sensitive info types) must exist.

1. **Author the policy in simulation/test mode first** — the Teams-message rule(s) and the SharePoint/OneDrive file rule(s), each keyed to your sensitive content, with the internal-vs- external recipient logic — **nothing blocked** yet.

1. **Validate on a pilot ring** — prove, on a small pilot user group, that the message and file activity is *seen* and *would* act; then move rules from simulation to **enforce** deliberately, with an explicit pass/fail gate and a documented rollback at every step.

The two coverage areas (how Teams DLP actually maps)

Area	Where the data lives	DLP location that governs it
Teams messages — 1:1 chat, group chat, standard & private channel posts	The Teams messaging service	**Teams chat and channel messages** location (a DLP location in its own right).
Files shared in a Teams chat	The **sender's OneDrive**	**OneDrive** location.
Files shared in a Teams channel	The **team's SharePoint site**	**SharePoint** location.

> ***This is the key insight most "Teams DLP" projects miss:*** *the **message** location and the **file** locations are **different**. A policy that only enables the Teams-messages location leaves **shared files uncovered**. This solution enables **all three** so the control is complete. **Verify the exact location names and Teams message-scope (chat vs channel vs private/shared channel) coverage against your licensing/tenant** — Teams DLP scope and the portal evolve.*

What Teams message DLP is (and is not)

• **It is** the extension of Purview Data Loss Prevention to the **Teams chat and channel messages** location — detecting sensitive content in messages and applying an action (block the message with a policy tip, differentiated for internal vs external recipients).

• **It is not** a retention, eDiscovery, or communication-compliance control. Those are separate Purview capabilities; this solution is **DLP** (preventing sensitive-data leakage), not records or conduct monitoring.

• **It is licensing-dependent.** Teams DLP (especially chat/channel-message DLP) is a premium capability — **verify the exact licensing required for message-level DLP against the Microsoft Purview service description for your tenant** (it typically requires E5-tier or equivalent compliance licensing, and message-level DLP has specific requirements). Some scope (e.g. private/ shared channels) and portal paths evolve — **verify against your licensing/tenant** rather than assuming (see `global-requirements-purview.md` §1).

Simulate first — these are high-impact controls

A **block** on a Teams message stops a real conversation at the moment someone hits send; a block on a shared file interrupts collaboration. A mis-scoped rule can block legitimate work. Therefore:

• The policy is deployed in **simulation/test mode first** (the DLP policy modes *Test with notifications* / *Test without notifications*; in the portal, *"Run the policy in simulation mode"*) — it **evaluates and records what it *would* do without blocking anyone** — validated on a small pilot user group, and only then moved to **enforce**.

Verify the exact simulation/test terminology in your tenant (the wording evolves).

> ***Determinations are not yours to make.*** **Which* data is sensitive, *whether* a given message or shared file is a policy violation, and *what* action a channel warrants are* ***compliance / records / security / HR-legal*** *determinations — this solution delivers the technical configuration and the evidence, not the determination (see* `global-requirements-purview.md` *§5).*

The standard deployment model

Every step follows the same repeatable model, with an explicit **pass/fail gate** between phases:

Assess → Design → Configure → Pilot (simulation) → Validate → Expand (enforce).

• **Assess** — confirm licensing, least-privilege roles, the in-scope sensitive content, the pilot user group, and compliance/security stakeholder sign-off.

• **Design** — decide the message action (block / policy tip) and the internal-vs-external logic, the file action, name every object, and set the pilot scope.

• **Configure** — build the DLP policy across the **Teams messages + SharePoint + OneDrive** locations in **simulation**, with the message and file rules.

• **Pilot** — run in simulation on the pilot users; confirm the message and file activity is seen and *would* act without blocking anyone.

• **Validate** — run the pass/fail gates in `05-testing.md`.

• **Expand** — move rules to enforce and widen scope **only** after the gate passes; roll back per `03-implementation.md` if a gate fails.

What's in the box

• This overview, a **Requirements** guide, an ordered **Implementation** guide, **Testing** (pass/fail gates), **Troubleshooting**, **Best Practices**, and a **Deliverables Summary** + **Executive Summary**.

• **Five PowerShell tools** (Security & Compliance PowerShell / Microsoft Graph): a **Discovery** inventory of existing DLP policies, the Teams/SharePoint/OneDrive coverage, labels, and SITs; a **Configure/Remediation** helper that builds the Teams-message + file DLP policy in **simulation (`-WhatIf` first)**; a **Validation** checker; a **Reporting** rollup of policy modes and DLP activity; and a **scope/health** helper — **read-only or `-WhatIf` by default; no credentials in code.**

Sources. *Microsoft Learn — *Learn about the default data loss prevention policy in Microsoft Teams*, *Data loss prevention and Microsoft Teams*, *Create and deploy data loss prevention policies* (locations, conditions, actions, simulation mode), and the *Microsoft Purview service description* (licensing for Teams message DLP). These capabilities are **licensing-dependent** and the Purview portal is **evolving — verify capability availability, the Teams message scope, portal paths, and cmdlet parameters against your licensing/tenant** rather than assuming.*

Requirements — Teams Chat & Channel DLP in Microsoft Purview

Confirm every prerequisite below **before** you configure anything. Category-level prerequisites (tenant, compliance licensing model, least-privilege roles, Security & Compliance PowerShell / Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, pilot scope + change control) are in `global-requirements-purview.md` — reference them; they are **not** repeated here. The items below are the ones **specific to Teams DLP**: the **message-level licensing**, the **three locations** (Teams messages, SharePoint, OneDrive), and the **internal-vs-external** control decision.

Each requirement is written as **Description · Why · Verify · Implement if missing.**

R1. Licensing that includes Teams message DLP, assigned to in-scope users

Description: The compliance licensing that includes **DLP for Teams chat and channel messages**, assigned to the in-scope users. **Verify the exact requirement against the Microsoft Purview service description for your tenant** — message-level Teams DLP is a premium capability (typically E5-tier / equivalent compliance licensing) and has its own licensing specifics distinct from file DLP. **Why:** Message-level Teams DLP is plan-gated; without the right licence the **Teams chat and channel messages** location and message actions are not available, and message DLP is licensed **per user**. **Verify:** Microsoft 365 admin center → **Billing** → **Licenses** confirms the SKU/add-on. In the Purview portal, **Data Loss Prevention** exposes the **Teams chat and channel messages** location. Cross-check against the *Microsoft Purview service description* — **verify against your licensing/tenant. Implement if missing:** Acquire and assign the required SKU/add-on to in-scope users before authoring the message rule. (File DLP via SharePoint/OneDrive may be available on lower tiers — but the message rule needs the message-DLP entitlement.)

R2. Least-privilege roles for DLP authoring

Description: The least-privilege roles, mapped to **named** people — **Compliance Administrator** or **Compliance Data Administrator** (or the **DLP Compliance Management** role group) to author the DLP policy and rules — with **Global Administrator** only where genuinely required. **Why:** Purview uses fine-grained role groups; policy authoring requires the matching role and should not be over-granted. **Verify:** Purview portal → **Roles & scopes / Permissions** confirms the DLP role membership. `DLPTEAMS-Discovery.ps1` reports the DLP policy/role signals it can read. **Implement if missing:** Assign the least-privilege DLP role under change control.

R3. Teams deployed with the in-scope chat/channel surfaces identified

Description: Microsoft Teams in use, with the **message surfaces** in scope identified — **1:1 and group chats**, **standard channel** messages, and (where covered by your licensing/tenant) **private channel** and **shared channel** messages. Whether **external / federated** participants are in scope must be stated. **Why:** Teams message DLP applies to specific surfaces; the design must state which are covered and whether external recipients are treated differently. Coverage of private/shared channels varies — **verify for your tenant. Verify:** Confirm in the Teams admin experience which surfaces are in use and whether external access/federation is enabled. **Verify the private/shared-channel DLP coverage against current Microsoft docs for your tenant. Implement if missing:** Document the in-scope surfaces and the external-participant posture with stakeholders before authoring rules.

R4. In-scope sensitive content to key the rules on (labels and/or SITs)

Description: The **sensitive content** the rules will match — **sensitivity labels** (e.g. *Confidential*) already published and in use, and/or **sensitive information types** (built-in or custom) and/or trainable classifiers — that define *what* triggers the message/file rules. **Why:** A DLP rule acts on **content that matches a condition**; without labels or SITs to match, the rule has nothing to key on (or would apply to *all* content, which is rarely intended). **Verify:** Purview → **Information Protection** → **Labels** shows the labels published; **Data classification** → **Sensitive info types** shows the SITs; in-scope content carries labels (Activity/ Content explorer where available). `DLPTEAMS-Discovery.ps1` inventories labels and SITs. **Implement if missing:** Publish/adopt the needed sensitivity labels or select the SITs first.

R5. All three DLP locations understood: Teams messages + SharePoint + OneDrive

Description: A clear understanding — and the intent to enable — the **three** locations a complete "Teams DLP" needs: **Teams chat and channel messages** (the message text), **SharePoint** (files shared in **channels**), and **OneDrive** (files shared in **chats**). **Why:** Enabling only the Teams-messages location leaves **shared files uncovered**, because Teams-shared files physically live in SharePoint/OneDrive. A complete control needs all three. **Verify:** Purview → **Data Loss Prevention** shows all three locations available for a policy. `DLPTEAMS-Discovery.ps1` reports which locations existing policies cover. **Verify the location names against your tenant. Implement if missing:** Plan the policy to span all three locations (or explicitly document a scope decision to cover messages only — but then say so, since files remain uncovered).

R6. Message action + internal-vs-external decision — agreed with stakeholders

Description: A documented decision, agreed with **compliance / security** (and HR/legal where the action affects users), of:

- The **message action** — block the message with a **policy tip**, notify only, and whether users can **override** with business justification.
- The **internal vs external** logic — e.g. audit-only for internal recipients but **block** when the content is shared with **external / everyone** (a common Teams DLP posture).
- The **file action** for SharePoint/OneDrive-shared files (block external sharing, notify, etc.).

Why: The action is a **policy decision, not a technical default**; blocking a Teams message or an external share has real operational impact and must reflect enterprise policy. **Verify:** Confirm the action matrix (internal vs external, message vs file) and the in-scope content **in writing** before you move any rule from simulation to enforce. **Implement if missing:** Capture the decision with the relevant owners; until then, deploy **audit / simulation only.** (See `global-requirements-purview.md` §5.)

R7. A named pilot user group and change control (simulation/test first)

Description: A **named pilot user group** (e.g. `SG-TeamsDLP-Pilot`) and agreement to run the policy in **simulation/test mode first**, under a **change-control** process, before any enforcing, broad-scope rollout. **Why:** Teams-message and file block actions are high-impact; simulation on a small ring proves the message/file activity is seen and the rules *would* act **without** disrupting real work, and lets you tune before enforcing. **Verify:** Confirm the pilot group exists, that **simulation/test mode** is used, and a change window/approval path. `DLPTEAMS-ScopeHealth.ps1` (`-WhatIf`) previews pilot-scope targeting. **Implement if missing:** Define the pilot group, deploy in simulation, and agree change control before enabling enforcement.

R8. Policy-tip / user-notification content prepared (where used)

Description: The **policy-tip text** and any **user notification / email** content that will show when the message rule fires (e.g. "This message contains sensitive information and was not delivered"), prepared and approved. **Why:** A clear, approved policy tip reduces friction and supports the lawful, transparent operation of the control; a confusing or absent tip generates tickets and workarounds. **Verify:** Confirm the policy-tip/notification text is drafted and approved by the relevant owners. **Implement if missing:** Draft the tip/notification content with compliance/communications before enforcing; simulation can run without it, but enforce should have it.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

Implementation Guide — Teams Chat & Channel DLP in Microsoft Purview

This is the ordered, do-it-in-this-sequence guide. It follows the standard model — **Assess** → **Design** → **Configure** → **Pilot (simulation/test)** → **Validate** → **Expand (enforce)** — with every object **named**, an explicit **pass/fail gate** at each phase, and a **rollback** for each change. Do **not** skip the simulation phase; Teams-message and file block actions are high-impact.

Named objects used throughout (rename to your convention, but be consistent):

Object	Name used here
DLP policy	`DLP-Teams-Pilot`
Teams-message rule	`DLP-Rule-TeamsMessages`
Shared-files rule (SharePoint + OneDrive)	`DLP-Rule-TeamsFiles`
Pilot user/security group	`SG-TeamsDLP-Pilot`

Locations this policy spans (R5): Teams chat and channel messages (message text), **SharePoint** (channel-shared files), **OneDrive** (chat-shared files).

> ***Tooling note.*** *The* ***policy and rules*** *can be authored in the* ***Purview portal or*** *with* `New-/Set-DlpCompliancePolicy` *and* `New-/Set-DlpComplianceRule`*; the provided* `DLPTEAMS-Configure.ps1` *builds them in* ***simulation*** *with* `-WhatIf` *by default. The Teams-messages location is set via the policy's Teams location parameter (e.g.* `-TeamsLocation`*), and files via* `-SharePointLocation` / `-OneDriveLocation`*.* ***Verify every portal path and cmdlet parameter against your tenant*** *— the portal and the Teams DLP scope evolve.*

Phase 0 — Assess (confirm you are ready)

1. **Confirm requirements.** Walk `02-requirements.md` R1–R8. Do **not** proceed until message-DLP

licensing, roles, the in-scope Teams surfaces, in-scope sensitive content, the three-location understanding, the action + internal/external decision, a pilot group, and the policy-tip content are all confirmed.

1. **Run Discovery.** `Connect-IPPSSession`, then `.\DLPTEAMS-Discovery.ps1` to inventory existing DLP

policies/rules, which locations they cover (Teams / SharePoint / OneDrive), sensitivity labels, and SITs. Keep the CSV as your baseline. (A **default Teams DLP policy** may already exist — note it.)

1. **Confirm pilot scope.** `.\DLPTEAMS-ScopeHealth.ps1` reports the pilot group membership (Graph)

and the current scope of any existing Teams policy.

Gate 0 (pass/fail): All of R1–R8 satisfied, Discovery baseline captured, pilot group defined. **Fail** → **stop** and close the gap; do not configure.

Phase 1 — Design the policy (locations, rules, actions)

1. **Confirm the three locations** the policy will span (R5): Teams messages + SharePoint + OneDrive,

scoped to the **pilot group** only (`SG-TeamsDLP-Pilot`).

1. **Confirm the content condition** (R4): the sensitivity label(s) and/or SIT(s) each rule keys on.
2. **Confirm the action matrix** (R6): message action (block + policy tip), internal-vs-external

logic (e.g. audit internal / block external), and the file action — **in writing**.

Gate 1: The location set, content condition, and action matrix are documented and approved. **Fail** → **stop**; the R6 decision is a determination owner's call, not the administrator's.

Phase 2 — Author the policy + rules in SIMULATION

Create **one** DLP policy, `DLP-Teams-Pilot`, spanning the three locations and scoped to the pilot group, in **simulation/test mode** (nothing is blocked). Then add the two rules. Author in the portal or run `.\DLPTEAMS-Configure.ps1 -WhatIf` first, then `.\DLPTEAMS-Configure.ps1 -Commit` (still simulation) to create the objects.

1. **Create the policy** (simulation, three locations, pilot scope):

• Locations: **Teams chat and channel messages**, **SharePoint**, **OneDrive**, each scoped to `SG-TeamsDLP-Pilot`.

• Mode: **simulation/test** (portal: *"Run the policy in simulation mode"*; PowerShell: a **Test** mode — `TestWithoutNotifications` for a silent pilot, or `TestWithNotifications` to also show policy tips). **Verify the exact mode value in your tenant.**

1. **Add the two rules**, each conditioned on your sensitive content:

Rule	Location(s)	Condition	Starting action	Notes
`DLP-Rule-TeamsMessages`	Teams messages	Sensitive label/SIT match	**Audit** (simulation); block-on-external per R6 in enforce	Differentiate internal vs external recipients per R6
`DLP-Rule-TeamsFiles`	SharePoint + OneDrive	Sensitive label/SIT match	**Audit** (simulation); block external share per R6 in enforce	Covers files shared in channels (SharePoint) and chats (OneDrive)

• Keep everything **Audit / notify-only** while the policy is in simulation — the *enforce* action (block / block-external) is applied in Phase 4, not now.

• Set the **internal vs external** condition on the message rule per R6 (e.g. a *recipient is external* / *content is shared with people outside the org* condition). **Verify the exact condition and action parameter names against** `Get-Help New-DlpComplianceRule -Full` **in your tenant** before scripting them.

1. **Save.** The policy is now evaluating on the pilot users **without blocking anyone.**

Gate 2: `DLP-Teams-Pilot` exists, is in **simulation/test**, spans **Teams + SharePoint + OneDrive**, scoped to the pilot, with both rules present and conditioned on sensitive content. `DLPTEAMS-Validate.ps1` reports **PASS** for policy-exists, three-locations, mode-is-simulation, scope-is-pilot, and two-rules-present. **Fail** → **rollback:** `Set-DlpCompliancePolicy -Identity DLP-Teams-Pilot -Mode Disable` (or delete with `Remove-DlpCompliancePolicy`); simulation blocks nothing, so impact is limited to telemetry.

Phase 3 — Pilot & validate in simulation

1. Let the policy run in simulation on `SG-TeamsDLP-Pilot` for the agreed observation window.
2. Perform the **sanctioned test actions** in `05-testing.md` as a pilot user (post a matching test string in a 1:1 chat and a channel, share a matching test file in a chat and a channel, and repeat with an **external** recipient where in scope).
1. Review **Activity explorer** and the policy's **simulation results**; run `.\DLPTEAMS-Report.ps1` to roll up policy modes and (where available) DLP activity.
1. **Tune:** adjust conditions to remove false positives and confirm both message and file activity is **seen**, and that the internal-vs-external logic behaves as designed.

Gate 3: The message rule and the file rule both registered the sanctioned test actions in simulation (would act), the internal-vs-external differentiation is observed, false positives triaged, and the **R6 action matrix** confirmed **in writing**. **Fail** → **do not enforce**; keep tuning in simulation. Rollback is unchanged from Phase 2 (disable/delete the policy).

Phase 4 — Enforce, then expand

Only after Gate 3 passes.

1. **Apply the enforce actions.** Update each rule to its **R6** action — block the message (with the approved **policy tip**, R8), block-on-external, and the file action — leaving **Audit** where the decision is audit-only.
1. **Move the policy out of simulation** to **enforce** (portal: turn the policy **On**; PowerShell: `Set-DlpCompliancePolicy -Identity DLP-Teams-Pilot -Mode Enable`). Do this **still scoped to the pilot** first.
1. **Re-run the sanctioned tests** (`05-testing.md`) and confirm the message is now actually blocked (with the policy tip) and the file share is handled as designed on the pilot.
1. **Expand scope in rings.** Widen the location scope (add the next user group), re-validate, and repeat. Consider renaming/cloning to a production policy (`DLP-Teams`) as you exit pilot.

Gate 4: On the pilot, the enforced message block/policy-tip and file action behave as designed, and audit-only paths record without blocking; expansion rings each pass the same gate before widening. **Fail** → **rollback:** immediately return the policy to **simulation/test** (`Set-DlpCompliancePolicy -Mode TestWithoutNotifications`) or **Disable** it — this stops enforcement at once — then diagnose per `06-troubleshooting.md` before re-enforcing.

Rollback summary (fast reference)

Situation	Action
A rule blocks legitimate messages/files	Set that rule's action back to **Audit**, or set the policy to **simulation/test**, then tune conditions.
The whole policy misbehaves	`Set-DlpCompliancePolicy -Identity DLP-Teams-Pilot -Mode Disable` (stops all enforcement) or `-Mode TestWithoutNotifications` (back to simulation).
A user should not be in scope	Remove them from `SG-TeamsDLP-Pilot` (or the scoped location membership).
Files covered but messages over-blocking (or vice-versa)	Disable the single offending rule (`Set-DlpComplianceRule -Identity <rule> -Disabled $true`) while leaving the other.
Start over cleanly	`Remove-DlpComplianceRule` for each rule, then `Remove-DlpCompliancePolicy -Identity DLP-Teams-Pilot`.

> *Enforcement changes are **high-impact** — make them under change control, keep the pilot ring small, and never move both message and file rules to **Block** at once without validating each in simulation first.*

Testing & Validation — Teams Chat & Channel DLP

This is the pass/fail gate set. Run it in two passes: **once in simulation** (Phase 3 — the message/ file is *seen* and the policy *would* act, nothing is blocked) and **again after enforce** (Phase 4 — the block actions *actually* apply). Every test uses **benign test content** deliberately marked to match your policy conditions (e.g. a message or document carrying your *Confidential* sensitivity label and/or seeded with a matching sensitive information type) — **never** real regulated data.

> ***Determination boundary.** These tests prove the **control works as configured**. Whether a given real message or shared file is a *violation*, and any consequence, is a **compliance / security / HR-legal** determination — not a test outcome (see `global-requirements-purview.md` §5).*

Run `.\DLPTEAMS-Validate.ps1` for the configuration gates and `.\DLPTEAMS-Report.ps1` for the activity/telemetry rollup between passes.

A. Configuration gates (automated — `DLPTEAMS-Validate.ps1`, read-only)

#	Check	PASS signal
A1	Policy exists	`DLP-Teams-Pilot` is present.
A2	Teams-messages location	The policy covers the **Teams chat and channel messages** location.

#	Check	PASS signal
A3	File locations	The policy also covers **SharePoint** and **OneDrive** (so Teams-shared files are governed).
A4	Simulation first	In the pilot pass, the policy **Mode** is a **Test/simulation** mode (not `Enable`).
A5	Pilot scope	The policy is scoped to `SG-TeamsDLP-Pilot`, **not** all users.
A6	Two rules present	`DLP-Rule-TeamsMessages` and `DLP-Rule-TeamsFiles` both exist and are enabled.
A7	Sensitive-content condition	Each rule carries a sensitivity-label and/or sensitive-info-type condition (not "all content").

Gate A: All applicable rows PASS. Any FAIL → fix in `03-implementation.md` before behavioural testing. **A3 (file locations) failing is the classic "messages covered, files uncovered" gap.**

B. Behavioural tests — messages and files (manual, as a pilot user)

Perform each action with a **matching** test message/file and record the result. In the **simulation pass** the expected result is *"activity recorded in Activity explorer / simulation results; no block."* In the **enforce pass** the expected result is the **R6** action for that path.

#	Path	Test action (as a pilot user, with matching test content)	Simulation PASS	Enforce PASS (per R6)
B1	**1:1 / group chat message**	Post the matching test string in a chat with an **internal** user	Activity recorded; no block	Handled per R6 (often audit/policy-tip for internal)
B2	**Channel message**	Post the matching test string in a standard channel	Activity recorded; no block	Handled per R6
B3	**Message to external**	Post the matching string in a chat/channel including an **external/federated** user (if in scope)	Activity recorded; no block	**Blocked** with policy tip per R6

#	Path	Test action (as a pilot user, with matching test content)	Simulation PASS	Enforce PASS (per R6)
B4	**File shared in chat (OneDrive)**	Share the matching test file in a 1:1 chat	Activity recorded; no block	Handled per R6 (e.g. block external share)
B5	**File shared in channel (SharePoint)**	Share the matching test file in a channel	Activity recorded; no block	Handled per R6

Gate B (simulation pass): Every in-scope path **B1–B5** recorded the activity. A path that records **nothing** is a coverage failure — do not enforce it; diagnose per `06-troubleshooting.md` (location not enabled, latency, condition match, licensing for message DLP).

Gate B (enforce pass): Every path enforces its **R6** action, the **internal-vs-external** differentiation behaves as designed, and the **policy tip** appears where configured. Any deviation → roll the policy back to simulation and tune.

C. Negative / false-positive tests (both passes)

#	Check	PASS signal
C1	Non-matching content	A message/file carrying **no** matching label/SIT triggers **no** DLP match.
C2	Internal allowed where designed	Where R6 permits internal sharing, an internal-only matching message is **not** blocked in the enforce pass.
C3	Out-of-scope user	A user **not** in `SG-TeamsDLP-Pilot` shows **no** enforcement from this policy.
C4	Override/justification	Where **override / business justification** is configured, the prompt appears and the override is **logged**.

Gate C: All applicable rows PASS. A false positive (C1/C2 fails) must be tuned **before** wider enforcement — false positives erode user trust and drive shadow workarounds.

D. Evidence to capture (for the defensibility pack)

Capture and retain, for each pass:

- `DLPTEAMS-Validate.ps1` output (configuration gate results, including the three-location check).
- `DLPTEAMS-Report.ps1` output (policy modes + DLP activity/incident export where available).

• Screenshots of **Purview** → **Activity explorer** showing message and file activity, and of the **policy tip / block notification** in a Teams message during the enforce pass.

• A completed **B-path matrix** signed off by the change owner, plus the **R6 action decision** and the confirmation that all **three locations** are enabled.

> *This evidence set is what makes the control **defensible** — it shows *what* was configured across messages **and** files, *that* each path was validated in simulation before enforcement, and *that* enforcement (including internal-vs-external) behaves as designed. Store it with the change record.*

Troubleshooting — Teams Chat & Channel DLP

The failures you will actually hit, with the most likely cause and the fix. Work top-down: **most "Teams DLP isn't working" reports trace back to a location not enabled, message-DLP licensing, condition matching, the internal-vs-external logic, or simulation-vs-enforce confusion — not the rule itself.** Where a portal path or cmdlet parameter is named, **verify it against your tenant** — the Purview portal and the Teams DLP scope evolve.

1. Messages are covered but shared FILES are not (the classic gap)

Symptoms: A sensitive **message** is caught, but a sensitive **file** shared in a chat or channel is not.

Fix

• **Enable the file locations.** Teams-shared files live in **SharePoint** (channel files) and **OneDrive** (chat files) — they are governed by those DLP locations, **not** the Teams-messages location. Add **SharePoint** and **OneDrive** to the policy. `DLPTEAMS-Validate.ps1` gate **A3** and `DLPTEAMS-Discovery.ps1` both flag this gap.

• Confirm the file rule (`DLP-Rule-TeamsFiles`) is present and enabled.

2. The Teams-messages location isn't available, or message DLP never fires

Symptoms: You cannot add the **Teams chat and channel messages** location, or message posts never generate a DLP match even in simulation.

Likely causes & fixes

• **Licensing.** Message-level Teams DLP is a premium, plan-gated capability with its own requirement (typically E5-tier / equivalent). **Verify against the Microsoft Purview service description for your tenant** (R1). Without the entitlement, the messages location / message actions are unavailable.

• **Surface not in scope.** Coverage of **private / shared channels** varies — verify which Teams surfaces your licensing/tenant covers (R3). A message posted in an unsupported surface won't match.

• **Propagation delay.** After creating/scoping the policy, allow time before message telemetry surfaces — it is **not** instant. Wait and retry.

3. The rule fires on the wrong things (false positives)

Symptoms: Legitimate messages/files are flagged/blocked.

Likely causes & fixes

- **Condition too broad.** If a rule keys on "all content" instead of a **sensitivity label / SIT**, it matches everything. Tighten the condition (R4). `DLPTEAMS-Validate.ps1` gate **A7** flags rules with no readable content condition.
- **SIT confidence/count too loose.** Raise the confidence level or instance count on the sensitive information type to cut noise, then re-test.
- **Internal traffic blocked when it should be allowed.** If the R6 intent was audit-internal / block-external, check the internal-vs-external condition on the message rule — an over-broad rule will block internal chats too.
- Tune **in simulation** — never tune by loosening an already-enforcing block in production.

4. The rule doesn't fire on content it should

Symptoms: A message/file you expect to match generates no DLP match.

Likely causes & fixes

- **Content doesn't carry the label/SIT.** The test message/file must actually carry the **sensitivity label **or contain the** sensitive information type** the rule keys on. Seed a known match.
- **Label not published / SIT not in scope.** Confirm the label is published to the user (Information Protection → Label policies) and the SIT exists (Data classification). `DLPTEAMS-Discovery.ps1` inventories both.
- **Rule disabled.** `DLPTEAMS-Validate.ps1` gate **A6b** flags disabled rules.
- **Wrong location.** A file shared in a channel is a **SharePoint** match, not a Teams-messages match — make sure you are looking at the right location's activity.

5. The internal-vs-external logic behaves unexpectedly

Symptoms: External sharing isn't blocked, or internal messages are blocked when they shouldn't be.

Likely causes & fixes

- **Recipient condition mis-set.** Confirm the message rule's *recipient is external / shared with people outside the org* condition matches the R6 intent. **Verify the exact condition parameter name for your tenant** (`Get-Help New-DlpComplianceRule -Full`).
- **Federation/guest settings.** Whether external participants are even possible depends on Teams external-access/federation settings — confirm with the Teams admin (R3).

- Re-test **in simulation** with a genuine external/guest participant before enforcing.

6. Simulation looks fine but enforcement doesn't block

Symptoms: In simulation the activity is recorded; after "turning it on" the block still doesn't happen.

Likely causes & fixes

- **Policy still in a Test mode.** Simulation records but does **not** block. To enforce, move the policy out of simulation: portal → turn the policy **On**; PowerShell → `Set-DlpCompliancePolicy -Identity DLP-Teams-Pilot -Mode Enable`. `DLPTEAMS-Validate.ps1` gate **A4** shows the current mode.
- **Rule action still Audit.** The rule may be **Audit / notify-only** even though the policy is enforcing. Set the message action to **block** (with policy tip) per R6.
- **User out of scope.** Enforcement only applies to users/locations in scope. Confirm with `DLPTEAMS-ScopeHealth.ps1` / gate **A5**.
- **Propagation delay.** Policy/action changes take time to apply. Wait, then retest.

7. `New-DlpCompliancePolicy` / location parameter errors

Symptoms: Creating the policy errors on a location parameter (`-TeamsLocation` / `-SharePointLocation` / `-OneDriveLocation`) or the scope value.

Fix

- **Verify the location parameter names and permitted values for your tenant.** They **evolve**. Run `Get-Help New-DlpCompliancePolicy -Full` and check current Microsoft docs, then adjust. The `DLPTEAMS-Configure.ps1` script deliberately **prints the plan** so you can verify before `-Commit`.
- Confirm the pilot group (`SG-TeamsDLP-Pilot`) exists and the identifier is one your tenant accepts (`DLPTEAMS-ScopeHealth.ps1`). Avoid `All` during the pilot (gate **A5**).
- **Prefer the portal for the first build** if PowerShell parameter names are uncertain, then read the result back with `DLPTEAMS-Discovery.ps1` to learn the exact shape in your tenant.

8. A default Teams DLP policy is interfering

Symptoms: DLP behaviour you didn't configure, or two policies acting on the same content.

Fix

- **Identify the default policy.** Tenants may have a **default Teams DLP policy**.

`DLPTEAMS-Discovery.ps1` lists all policies and their locations/modes so you can see what is already acting. Reconcile your pilot policy with it (do not double-apply conflicting actions); adjust or scope the default per your change control if needed.

9. Activity report is empty or export fails

Symptoms: `DLPTEAMS-Report.ps1` returns no activity, or the Activity Explorer export warns and skips.

Likely causes & fixes

• **Licensing/permission.** `Export-ActivityExplorerData` is licensing-dependent; the script **warns

and degrades to policy-modes only **rather than failing. Fall back to** Purview → Activity explorer **/** DLP reports** in the portal.

• **Quiet pilot window.** A small pilot may have no activity in the window — widen `-Days` or perform

the sanctioned test actions from `05-testing.md` first.

• **Role.** Ensure the connected account holds a role that can read Activity Explorer.

10. "It blocked a legitimate conversation / share" — emergency rollback

Do this immediately, then diagnose:

• Return the policy to **simulation** or **off**: `Set-DlpCompliancePolicy -Identity DLP-Teams-Pilot

-Mode TestWithoutNotifications `(simulation) or` -Mode Disable` (off). Either **stops enforcement at once.**

• Or disable the single offending rule: `Set-DlpComplianceRule -Identity <rule> -Disabled $true`.

• Then work the relevant section above (usually §3 false positives or §5 external logic), fix **in

simulation**, re-validate with `DLPTEAMS-Validate.ps1`, and only then re-enforce.

When to escalate to the determination owners (not a technical fix)

Some questions are **not** troubleshooting — they are **determinations** that belong to compliance / security / records / HR-legal (see `global-requirements-purview.md` §5):

• *Should* external sharing be **blocked** or only **audited**? → the **R6** decision owners.

• Is *this content* actually sensitive / in scope? → **compliance / data owners**.

• Is a user's message/share a **violation**, and what is the consequence? → **HR / legal**.

Provide them the **evidence** (`DLPTEAMS-Report.ps1`, Activity explorer, the validation output) — not a verdict.

Best Practices — Teams Chat & Channel DLP

Field-tested guidance for running Teams DLP well. It complements — does not replace — the implementation steps and the pass/fail gates.

Cover messages AND files — the single most common gap

- **Enable all three locations.** A "Teams DLP" policy that only enables the **Teams chat and channel

messages **location leaves** shared files uncovered **— those files live in** SharePoint **(channel files) and** OneDrive **(chat files). Always enable** Teams messages + SharePoint + OneDrive **so the control is complete.** `DLPTEAMS-Validate.ps1` **gate** A3** exists precisely to catch this.

- **Say so if you deliberately scope to messages only.** If files are out of scope for a phase,

document it — otherwise stakeholders will assume "Teams DLP" covered the confidential file someone dropped in a channel.

Deploy the way the house model says — simulate, then enforce

- **Never enforce a rule you have not validated in simulation.** The most common cause of a Teams DLP

incident is a **block** moved to enforce without a simulation pass. Every rule goes **Audit/simulation** → **validate** → **enforce**, one ring at a time.

- **Small pilot ring first.** Scope a pilot user group (`SG-TeamsDLP-Pilot`) before the tenant. It is

far cheaper to find a false positive on ten users than ten thousand.

- **Enforce messages and files separately.** Do not flip both rules to block together — enforce the

higher-risk path first (often external message sharing), validate, then the other.

- **Keep a rollback ready.** `Set-DlpCompliancePolicy -Mode TestWithoutNotifications` (back to

simulation) or `-Mode Disable` stops enforcement instantly — know the command before you enforce.

Get the internal-vs-external logic right

- **Differentiate recipients deliberately.** A very common, proportionate posture is **audit / policy

tip for internal **sharing but** block when shared externally / with everyone**. Decide this with compliance (R6) — it balances protection against disrupting normal internal collaboration.

- **Treat external/federated participants as the higher-risk path.** Sensitive content leaving to a

guest or federated tenant is the exposure most Teams DLP programs exist to stop.

Key the rules on *sensitive* content, not everything

- **Condition on labels and/or SITs**, not "all content." A Teams rule that blocks *any* message with,

say, a number pattern will generate noise and workarounds; one that blocks **labeled/classified** content is defensible and proportionate.

- **Lead with sensitivity labels where they exist** — a label is an explicit business decision about

sensitivity and travels with the file. Fall back to **sensitive information types** where labels are not yet adopted, and tune SIT confidence/count to reduce false positives before you enforce.

Write a clear policy tip

- **A good policy tip reduces tickets.** Tell the user *what* happened and *what to do* ("This message

contains sensitive information and wasn't delivered to external recipients; remove it or contact compliance"). Get it approved (R8) before enforce.

- **Use override/business-justification thoughtfully** where the business genuinely needs a path — and

make sure the override is **logged** for the evidence pack.

Operate it, don't set-and-forget

- **Review Activity explorer and DLP reports regularly** and roll up with `DLPTEAMS-Report.ps1`.

Rising match volume on a path is a signal to review scope, tuning, or user education — not automatically a reason to block harder.

- **Mind the default Teams DLP policy.** Tenants may have a **default** Teams DLP policy — know whether

it exists and how it interacts with yours (`DLPTEAMS-Discovery.ps1` lists policies and their locations) so you do not double-apply or conflict.

- **Version and name policies clearly.** Move from `DLP-Teams-Pilot` to a production `DLP-Teams` as you

exit pilot; keep a change log of scope and action changes.

- **Re-validate after portal changes.** The Purview portal and the Teams DLP scope (e.g. private/

shared channel coverage) evolve; re-run `DLPTEAMS-Validate.ps1` after Microsoft feature updates and confirm nothing regressed.

Keep the roles and determinations where they belong

- **Least privilege.** DLP authoring uses a specific compliance role — do not run it from a standing

Global Administrator session.

- **Determinations stay with the owners.** *Which* data is sensitive, *which* paths to block, and

whether a message/share is a violation are **compliance / security / HR-legal** calls. You provide the configuration and the evidence, not the judgement (see `global-requirements-purview.md` §5).

- **Communicate before you enforce.** Policy tips and user notice reduce friction and support the

lawful, transparent operation of a Teams control; surprise blocks generate tickets and workarounds.

Security & handling of the tooling

- **Read-only /** `-WhatIf` **by default.** Run the provided scripts in their safe default first; only add

`-Commit`/`-Apply` when you intend to write, under change control.

- **No secrets in code.** The scripts use interactive modern-auth (`Connect-IPPSSession` /

`Connect-MgGraph`); never hardcode credentials, tokens, or app secrets.

• **Least-privilege connections.** Connect with the least-privilege role that the task needs, not a standing Global Administrator session.

Deliverables Summary — Teams Chat & Channel DLP in Microsoft Purview

Everything included in this solution, what each piece is for, and how it maps to the deployment model. Use it as the table of contents and the "did we finish?" checklist.

Documents

Document	What it gives you
01 — Solution Overview	What Teams DLP delivers, the two coverage areas (messages via the Teams-messages location; **files via SharePoint/OneDrive**), the simulate-first rule, and the Assess→Design→Configure→Pilot→Validate→Expand model.
02 — Requirements	The solution-specific prerequisites (R1–R8): message-DLP licensing, DLP roles, in-scope Teams surfaces, in-scope sensitive content, the three-location understanding, the action + internal/external decision, a pilot group, and policy-tip content — each with Verify / Implement-if-missing. References the category `global-requirements-purview.md`.
03 — Implementation Guide	The ordered build: design → author the policy across **Teams messages + SharePoint + OneDrive** with two rules **in simulation** → pilot & validate → enforce & expand. Every object named, with a pass/fail gate and rollback per phase.
05 — Testing & Validation	The pass/fail gates: configuration gates (A, including the three-location check), message + file behavioural tests (B) run in simulation then enforce, false-positive/negative tests (C), and the evidence to capture (D).
06 — Troubleshooting	The failures you will actually hit — messages covered but files not, message DLP not firing (licensing), external logic wrong, simulation-vs-enforce confusion — each with cause and fix.
07 — Best Practices	How to run it well: cover messages AND files, simulate-then-enforce discipline, the internal-vs-external logic, keying on sensitive content, a clear policy tip, ongoing operation, and role/determination boundaries.

Document	What it gives you
Executive Summary	The one-page business-level summary for stakeholders and sign-off.

PowerShell tools (5)

All are **read-only or `-WhatIf` by default**, use interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`), and contain **no credentials**. They do **not** invent cmdlets.

Script	Purpose	Safe default
DLPTEAMS-Discovery.ps1	Inventory existing DLP policies/rules and which locations they cover (**Teams / SharePoint / OneDrive**), sensitivity labels, and sensitive info types — the baseline, including any default Teams DLP policy.	Read-only
DLPTEAMS-Configure.ps1	Build/update the `DLP-Teams-Pilot` policy across the three locations and the message + file rules **in simulation**; requires `-Commit` to write and supports `-WhatIf`.	`-WhatIf` (preview)
DLPTEAMS-Validate.ps1	Run the configuration gates (policy exists, Teams-messages location, **SharePoint+OneDrive file locations**, simulation-first, pilot scope, two rules, sensitive-content conditions) with PASS/FAIL output.	Read-only
DLPTEAMS-Report.ps1	Roll up policy modes and, where available, **DLP activity** via Activity-explorer export — the operational + evidence view.	Read-only
DLPTEAMS-ScopeHealth.ps1	Report pilot-scope health (pilot-group membership via Graph, policy scope) and preview pilot-scope targeting; `-WhatIf` for any scoping change.	`-WhatIf` / read-only

How it maps to the deployment model

Phase	Deliverables used
Assess	01, 02, `DLPTEAMS-Discovery.ps1`, `DLPTEAMS-ScopeHealth.ps1`

Phase	Deliverables used
Design	02, 03 (named objects + action matrix), 07
Configure	03, `DLPTEAMS-Configure.ps1 -WhatIf` then `-Commit` (simulation)
Pilot (simulation)	03, 05 (Gate B simulation pass), `DLPTEAMS-Report.ps1`
Validate	05, `DLPTEAMS-Validate.ps1`, `DLPTEAMS-Report.ps1`
Expand (enforce)	03 (Phase 4), 05 (Gate B enforce pass), 06, 07

Definition of done

• `DLP-Teams-Pilot` built **in simulation** spanning **Teams messages + SharePoint + OneDrive**, all configuration gates (A) PASS — including the three-location check.

• Both rules (`DLP-Rule-TeamsMessages`, `DLP-Rule-TeamsFiles`) present, conditioned on sensitive content, with the internal-vs-external logic on the message rule.

• Behavioural gates (B) PASS in the **simulation pass**, then in the **enforce pass** for the paths the R6 decision blocks; false-positive gates (C) PASS.

• **Evidence pack** (D) captured and stored with the change record.

• Regulatory / policy determinations (which data, which action, internal vs external) recorded by the **compliance / security** owners — not by the administrator.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLPTEAMS-Configure.ps1

Purpose: Creates (or updates) a Teams DLP compliance policy spanning the Teams-messages, SharePoint, and OneDrive locations and scoped to a pilot group, in SIMULATION/TEST mode, plus two rules — one for Teams messages and one for shared files (SharePoint+OneDrive) — each conditioned on sensitive content and set to Audit while in simulation. SAFE BY DEFAULT: previews with -WhatIf and writes NOTHING unless you pass -Commit. It never enforces (never sets Block/Enable) — moving to enforce is a deliberate step in Phase 4 of the implementation guide. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPTEAMS-Configure.ps1 -Commit # create the policy + rules in SIMULATION .\DLPTEAMS-Configure.ps1 -Commit -PilotGroup "SG-TeamsDLP-Pilot" -LabelName "Confidential" EXPECTED RESULTS With -Commit: policy 'DLP-Teams-Pilot' exists in a Test/simulation mode across Teams+SharePoint+ OneDrive, pilot scope, with two Audit rules. Validate with DLPTEAMS-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The location parameter names (-TeamsLocation/-SharePointLocation/-OneDriveLocation), the internal-vs-external condition/action names, and the simulation Mode value EVOLVE — the script uses documented names and PRINTS the plan for you to verify with `Get-Help New-DlpComplianceRule -Full` against your tenant before committing. It does NOT enforce (no Block/Enable). > [CmdletBinding()] param([switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "DLP-Teams-Pilot", [string]$MsgRuleName = "DLP-Rule-TeamsMessages", [string]$FileRuleName= "DLP-Rule-TeamsFiles", [string]$PilotGroup = "SG-TeamsDLP-Pilot", # pilot user/security group for all three locations [string]$LabelName = "Confidential", # sensitivity label the rules key on (adjust to your taxonomy) [string]$SimMode = "TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPTEAMS-Configure.ps1 -Commit # create the policy + rules in SIMULATION .\DLPTEAMS-Configure.ps1 -Commit -PilotGroup "SG-TeamsDLP-Pilot" -LabelName "Confidential" EXPECTED RESULTS With -Commit: policy 'DLP-Teams-Pilot' exists in a Test/simulation mode across Teams+SharePoint+ OneDrive, pilot scope, with two Audit rules. Validate with DLPTEAMS-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The location parameter names (-TeamsLocation/-SharePointLocation/-OneDriveLocation), the internal-vs-external condition/action names, and the simulation Mode value EVOLVE — the script uses documented names and PRINTS the plan for you to verify with `Get-Help New-DlpComplianceRule -Full` against your tenant before committing. It does NOT enforce (no Block/Enable). > [CmdletBinding()] param([switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "DLP-Teams-Pilot", [string]$MsgRuleName = "DLP-Rule-TeamsMessages", [string]$FileRuleName= "DLP-Rule-TeamsFiles", [string]$PilotGroup = "SG-TeamsDLP-Pilot", # pilot user/security group for all three locations [string]$LabelName = "Confidential", # sensitivity label the rules key on (adjust to your taxonomy) [string]$SimMode =

"TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant)

DLPTEAMS-Discovery.ps1

Purpose: Read-only baseline discovery for Teams DLP. Inventories the DLP compliance policies and rules already in the tenant, flags for each whether it covers the Teams-messages, SharePoint, and OneDrive locations (so you can see whether Teams messages AND shared files are governed), reports each policy's mode (simulation/test vs enable), and lists the sensitivity labels and sensitive information types the rules could key on. Also surfaces any DEFAULT Teams DLP policy. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Discovery.ps1 .\DLPTEAMS-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPTEAMS-Discovery.csv) of DLP policies (with Teams/SharePoint/OneDrive location flags + Mode), DLP rules, and labels; with -IncludeSensitiveTypes, sensitive info types to .\DLPTEAMS-SensitiveTypes.csv. A policy that covers Teams messages but NOT SharePoint/OneDrive is flagged as a files-uncovered gap. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Teams message DLP is licensing-dependent — the script WARNS where a feature is not readable rather than assuming. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPTEAMS-Discovery.csv", [string]$SensitiveOutPath = ".\DLPTEAMS-SensitiveTypes.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Discovery.ps1 .\DLPTEAMS-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPTEAMS-Discovery.csv) of DLP policies (with Teams/SharePoint/OneDrive location flags + Mode), DLP rules, and labels; with -IncludeSensitiveTypes, sensitive info types to .\DLPTEAMS-SensitiveTypes.csv. A policy that covers Teams messages but NOT SharePoint/OneDrive is flagged as a files-uncovered gap. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Teams message DLP is licensing-dependent — the script WARNS where a feature is not readable rather than assuming. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPTEAMS-Discovery.csv", [string]$SensitiveOutPath = ".\DLPTEAMS-SensitiveTypes.csv")

DLPTEAMS-Report.ps1

Purpose: Read-only operational + evidence report for Teams DLP. Summarizes the DLP policies that cover the Teams-messages / SharePoint / OneDrive locations and their mode (simulation vs enforce), and — where the tenant/licensing exposes it — exports recent DLP activity from Activity Explorer so you can see the message/file matches the policy is recording. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Report.ps1 .\DLPTEAMS-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\DLPTEAMS-PolicyModes.csv (policies + mode + Teams/SPO/OneDrive flags) and, where available, .\DLPTEAMS-Activity.csv (Activity Explorer DLP events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns) rather than assuming availability. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPTEAMS-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPTEAMS-Activity.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Report.ps1 .\DLPTEAMS-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\DLPTEAMS-PolicyModes.csv (policies + mode + Teams/SPO/OneDrive flags) and, where available, .\DLPTEAMS-Activity.csv (Activity Explorer DLP events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns) rather than assuming availability. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPTEAMS-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPTEAMS-Activity.csv")

DLPTEAMS-ScopeHealth.ps1

Purpose: Scope and health helper for Teams DLP. Summarizes the pilot user/security group membership from Microsoft Graph and shows the current Teams/SharePoint/OneDrive scope of the Teams DLP policy — so you can see what the policy WOULD cover before you widen it. Read-only by default; any scope change is preview-only via -WhatIf (this tool does not itself widen scope — it prints the command for you to run under change control). HOW TO RUN PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\DLPTEAMS-ScopeHealth.ps1 .\DLPTEAMS-ScopeHealth.ps1 -PilotGroup "SG-TeamsDLP-Pilot" -WhatIf EXPECTED RESULTS A console summary: pilot-group membership (Graph) and the policy's current Teams/SharePoint/OneDrive scope. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Microsoft Graph (Get-MgGroup / Get-MgGroupMember) and Get-DlpCompliancePolicy; none invented. > [CmdletBinding()] param([switch]$WhatIf, [string]$PilotGroup = "SG-TeamsDLP-Pilot", [string]$PolicyName = "DLP-Teams-Pilot")

How to run: PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\DLPTEAMS-ScopeHealth.ps1 .\DLPTEAMS-ScopeHealth.ps1 -PilotGroup "SG-TeamsDLP-Pilot" -WhatIf EXPECTED RESULTS A console summary: pilot-group membership (Graph) and the policy's current Teams/SharePoint/OneDrive scope. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Microsoft Graph (Get-MgGroup / Get-MgGroupMember) and Get-DlpCompliancePolicy; none invented. > [CmdletBinding()] param([switch]$WhatIf, [string]$PilotGroup = "SG-TeamsDLP-Pilot", [string]$PolicyName = "DLP-Teams-Pilot")

DLPTEAMS-Validate.ps1

Purpose: Read-only validation of the Teams DLP policy against the configuration gates in 05-testing.md (Gate A). Confirms the policy exists, covers the Teams-messages location AND the SharePoint+OneDrive file locations (the common "messages covered, files uncovered" gap), is in a Test/simulation mode (during the pilot pass), is scoped to a pilot (not all users), and has the two expected rules present and enabled with a sensitive-content condition. Prints PASS/FAIL per check. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Validate.ps1 .\DLPTEAMS-Validate.ps1 -PolicyName "DLP-Teams-Pilot" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A4 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard

Get-DlpCompliance* cmdlets; none invented. Whether the policy SHOULD be in simulation depends on the phase — the script reports the mode and lets you judge against the phase you are in. > [CmdletBinding()] param([string]$PolicyName = "DLP-Teams-Pilot", [string[]]$ExpectedRules = @("DLP-Rule-TeamsMessages","DLP-Rule-TeamsFiles"))

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPTEAMS-Validate.ps1 .\DLPTEAMS-Validate.ps1 -PolicyName "DLP-Teams-Pilot" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A4 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-DlpCompliance* cmdlets; none invented. Whether the policy SHOULD be in simulation depends on the phase — the script reports the mode and lets you judge against the phase you are in. > [CmdletBinding()] param([string]$PolicyName = "DLP-Teams-Pilot", [string[]]$ExpectedRules = @("DLP-Rule-TeamsMessages","DLP-Rule-TeamsFiles"))